SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 – Introduction to Artificial Intelligence

PRN: 25UAM143 **Name:** TAABISH SHAIKH

Division: B **Date:** 04-Oct-2026 **GitHub / Portfolio Link (optional):** _____

1. System Title & Short Description

8-Puzzle Solver System (BFS / DFS / A*) – continued from SLE-1 (code) and SLE-2 (profiling and comparison). The system takes a 3x3 sliding-tile board (tiles 1–8 and a blank) and a goal board, and finds the sequence of blank moves (Up/Down/Left/Right) that turns the start into the goal. It first checks that the puzzle is solvable, then runs the search algorithm chosen by the user. It reports the moves, solution cost, nodes explored, time and memory, so BFS, DFS and A* can be compared as in SLE-2.

2. Context Diagram (Level 1)

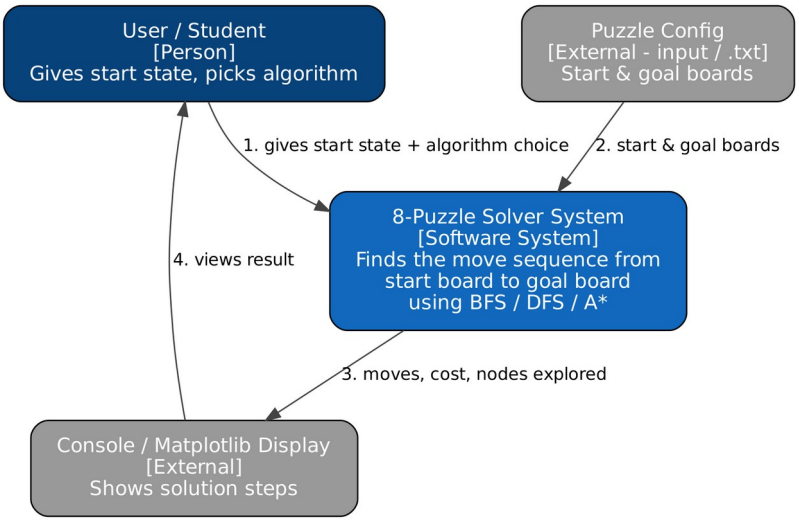


Figure 1: System Context – 8-Puzzle Solver System

The User gives the start board and picks BFS, DFS or A*. The 8-Puzzle Solver System is the single central system. The start and goal boards come from the program input or a small text file, and the solution (moves, cost, nodes explored) is shown on the console. There are no other external services, so the system stays simple.

3. Container Diagram (Level 2)

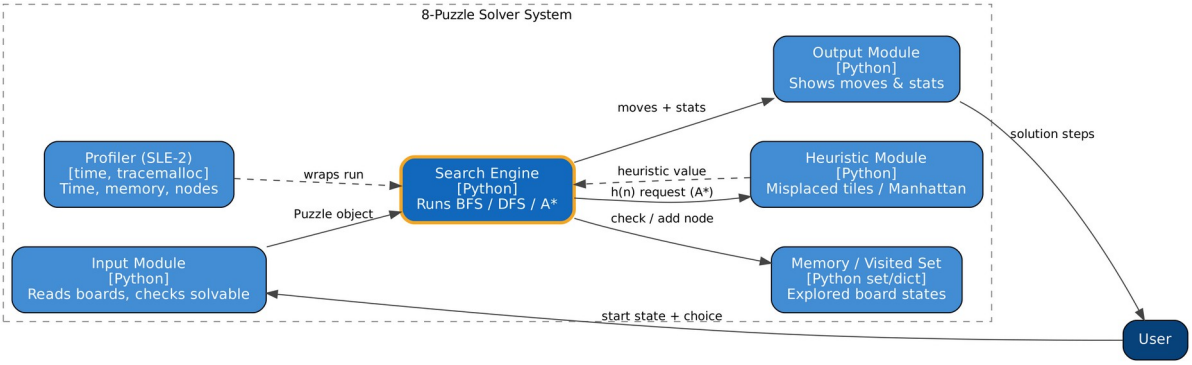


Figure 2: Containers inside the 8-Puzzle Solver System (6 containers)

- **Input Module:** reads the start and goal boards, validates them (tiles 0–8 used once) and checks solvability by counting inversions.
- **Search Engine:** core of the system; runs BFS, DFS or A* on the puzzle and returns the move sequence and statistics.
- **Heuristic Module:** gives h(n) (misplaced tiles or Manhattan distance); used only by A*.
- **Memory / Visited Set:** stores already-explored board states so the search never repeats a state or loops.
- **Output Module:** prints the solution move by move (board after each step) with cost and nodes explored.
- **Profiler (from SLE-2):** wraps a search run and measures time and memory for the comparison report.

4. Component Diagram (Level 3) – Search Engine

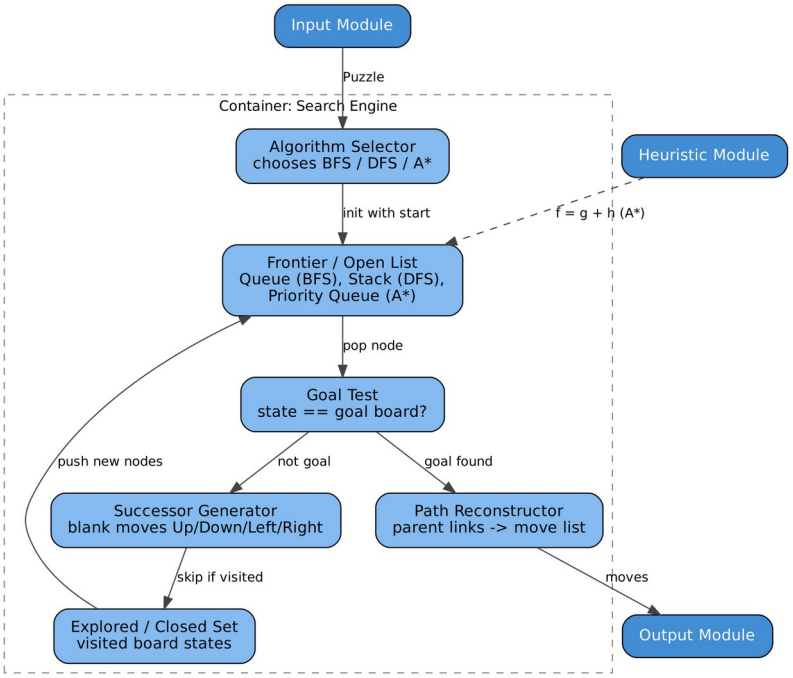


Figure 3: Components inside the Search Engine container

The Algorithm Selector picks the strategy. The Frontier is a queue for BFS, a stack for DFS and a priority queue for A* – this is the only part that really changes between algorithms. Each loop pops a state, the Goal Test compares it with the goal board, and the Successor Generator makes new boards by sliding the blank Up/Down/Left/Right. The Explored Set filters repeated boards. When the goal is reached, the Path Reconstructor follows parent links back to the start to build the move list.

5. Code Level Overview (Level 4)

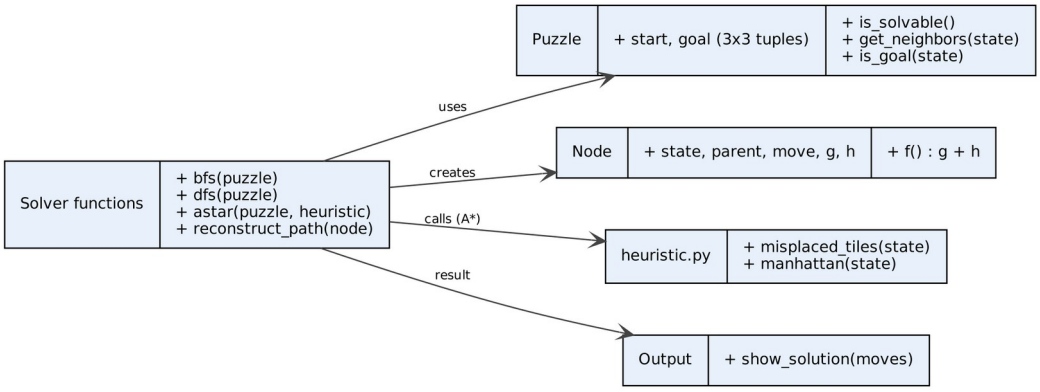


Figure 4: Main classes and functions (names only)

- **class Node :** stores board state, parent, move, cost g and heuristic h; f() = g + h.
- **class Puzzle :** holds start and goal boards; is_solvable(), get_neighbors(state), is_goal(state).
- **def bfs() / def dfs() :** uninformed search using a queue / stack with an explored set.
- **def astar() :** informed search using a priority queue ordered by f = g + h.
- **def misplaced_tiles() / manhattan() :** heuristic functions used by astar().
- **def reconstruct_path() :** walks parent links from the goal back to the start and reverses the list of moves.
- **def show_solution() :** prints the board after every move.

6. Design Decisions

The system is split by responsibility so that one part can be changed without touching the others: the Heuristic Module is separate so Manhattan and misplaced tiles can be swapped without editing the search; the Visited Set is separate because the 8-puzzle has many repeated states (up to 181,440 reachable boards) and loops are otherwise certain. BFS, DFS and A* share the same loop and differ only in the Frontier type, which makes the

comparison in SLE-2 fair. The Profiler is kept outside the Search Engine so measuring does not change the algorithm. I kept to 6 containers to keep the design readable.

7. AI Contribution Note

- **AI tools used (if any):** Claude (Anthropic) – used to draft the C4 diagrams (Graphviz) and the first version of this report.
- **What AI helped with:** suggesting the container split, drawing the diagrams, and wording the explanations.
- **What I did myself:** chose the 8-puzzle from my SLE-1/SLE-2 work, checked that the containers and class names match my own code, and edited the text to match my implementation. [Edit this line to match what you actually did.]

8. Conclusion

C4 helped me look at the same 8-puzzle solver at four zoom levels: who uses it, what the main parts are, how the Search Engine works inside, and which classes and functions implement it. I learned that good architecture means each part has one clear job, and that BFS, DFS and A* are really one search loop with a different Frontier and an optional heuristic. Keeping the diagrams small made the design easier to explain and to change.